

AIMLCZG546 · SESSION 6 · COMPANION READER

Event-Driven Architecture & ML Design Patterns

Model Registry, Batch Serving, and Real-Time Inference Explained

Session 6 · Read alongside the lecture slides · Every pattern and concept worked through in full

How to use this companion. **Blue** = intuition. **Amber** = everyday analogy. **Green** = worked scenario. **Red** = pitfall. **Purple** = one line to remember.

1 Event-Driven Architecture

Event-Driven Architecture (EDA) is an architectural style in which components communicate by producing and consuming *events* rather than calling each other directly.

Context: A complex system handles asynchronous events from user actions, external systems, or internal processes. Services must react to these events without being tightly coupled to each other.

Problem: If Service A directly calls Service B, they are coupled. If Service B is slow or down, Service A is blocked. If you add Service C (also interested in the same events), you must modify Service A again.

Solution: Introduce a **message broker** between producers and consumers.

The intuition

Imagine a school PA system. The principal (publisher) makes one announcement. Every classroom (subscriber) hears it simultaneously. The principal does not know or care which classrooms exist or how they react. Adding a new classroom does not require the principal to change anything.

1.1 Publish-Subscribe Model

- **Publishers** (Producers) — generate events and publish them to the broker without knowing who will consume them.
- **Message Broker** — stores, routes, and delivers events. Examples: Apache Kafka, RabbitMQ, Amazon SQS, Azure Service Bus.
- **Subscribers** (Consumers) — declare interest in specific event types; the broker delivers matching events.

Worked example: Fraud detection with EDA

A transaction occurs at 14:32:01 for customer C491, amount \$1,200, merchant: “Electronics Ltd, Lagos.”

Step 1 — Payment Service (Publisher) publishes event to Kafka topic `transactions.created`:

```
{"id": "txn-9921", "customer": "C491", "amount": 1200, "merchant": "Electronics Ltd", "country": "NG"}
```

Step 2 — Fraud Detection Service (Subscriber) consumes the event. Runs XGBoost model. Fraud probability = 0.94. Publishes to topic `fraud.detected`:

```
{"txn_id": "txn-9921", "fraud_prob": 0.94, "action": "BLOCK"}
```

Step 3 — Notification Service (Subscriber to `transactions.created`) sends SMS: “A transaction of \$1,200 in Nigeria was attempted. Was this you?”

Step 4 — Analytics Service (Subscriber to `transactions.created`) updates the fraud dashboard.

Why EDA wins here: The Payment Service published one event. Three services reacted independently. Adding a fourth (e.g., a compliance logging service) requires zero changes to existing services — just subscribe to the topic.

Where this shows up in ML/AI. ML model retraining pipelines are naturally event-driven: “data drift detected” event triggers retraining; “new labelled batch available” event triggers evaluation; “model deployed” event triggers A/B test initialisation.

2 Model Registry Pattern

2.1 The Registry Pattern (Software Engineering)

A **registry** is a central hub that dynamically stores, locates, and retrieves class instances or services. It acts as a directory: “where is the thing I need?” Combined with **version control**, it becomes a model registry.

2.2 What a Model Registry Stores

Git (code version control)	ML Model Registry
Source code files (.py, .java)	Trained model files (.pkl, .pt, .onnx)
Documentation (.md, .rst)	Model metrics (accuracy, F1, AUC by dataset version)
Configuration files	Hyperparameters and training configuration
Commit messages	Experiment run description and notes
Branch/tag names	Model aliases (staging, production, champion)
—	Dataset version and data lineage
—	Training environment (Python version, package versions)

2.3 MLflow Registry in Practice

MLflow is the most widely used open-source ML registry. Its four core capabilities:

1. **Version Control** — every trained model is saved as a versioned artefact. Multiple versions co-exist; you can compare, roll back, or deploy any version.
2. **Lineage and Traceability** — each model version links to the exact training run (data

version, hyperparameters, metrics, code commit).

3. **Production Workflows** — **aliases** and **tags** control the promotion pipeline: a model moves from `@staging` to `@production` only after evaluation passes.
4. **Governance** — access control, audit log, metadata for regulatory compliance.

Worked example: Churn model lifecycle in MLflow

Day 1: Data scientist trains v1 (logistic regression). Logs to MLflow: accuracy=0.76, AUC=0.79. Registers as version 1. Tag: `status=experimental`.

Day 5: Trains v2 (XGBoost, new features). Logs: accuracy=0.84, AUC=0.88. Registers as version 2. Sets alias: `@staging`.

Day 7: QA team runs offline evaluation on holdout set. Version 2 passes. Sets alias: `@production`. Version 1 alias cleared.

Day 30: New team trains v3 (LightGBM, fresh data). Registers as version 3. Sets alias: `@staging`. Begins A/B test in production (5 % traffic to v3).

Day 37: A/B test complete. Version 3 reduces churn by 8 % vs version 2. Sets `@production` to version 3. Version 2 demoted.

Rollback scenario: Version 3 causes a p95 latency spike in production. Team runs: `mlflow.set_registered_model_alias("churn-model", "production", "2")`. Traffic immediately routed back to version 2. Total rollback time: 45 seconds.

3 Batch vs Real-Time Serving

Two fundamentally different patterns for deploying ML models in production. The choice depends on latency requirements, throughput, cost, and business context.

3.1 Batch Prediction (Offline)

The model runs periodically over a large dataset and saves predictions for later consumption.

★ Everyday picture

A newspaper is printed once per day with tomorrow's weather forecast. Readers get the information with a delay, but the cost of printing is amortised over millions of readers. You can't ask the newspaper to predict tomorrow's weather right now, at this exact second.

When to use Batch:

- Customer segmentation: group users by weekly behaviour → email campaign.
- Credit risk scoring at end of business day.
- Product demand forecasting for warehouse stocking (run nightly).
- Recommendation emails: score all users at 2am, send emails at 9am.

3.2 Online Prediction (Real-Time)

The model is deployed as a persistent REST API and returns predictions in milliseconds, on demand.

★ Everyday picture

Weather radar: the moment you open the app, it queries the satellite and shows you what is happening *right now*. Instant response is the point; a 24-hour delay is useless.

When to use Online:

- Fraud detection: must block a transaction *before* it is authorised.
- Dynamic pricing: ride-share surge pricing changes every few seconds.
- Spam filtering: classify before the email lands in the inbox.
- Chatbot responses: user is waiting.

3.3 Comparison

Property	Batch	Online
Latency	Minutes to hours	Milliseconds to seconds
Throughput	Very high (process millions per run)	Lower (request-by-request)
Cost	Low (run only when needed)	Higher (server always running)
Data freshness	Stale (predictions age between runs)	Real-time (uses current data)
Trigger	Scheduled (cron, Airflow)	Per request
Infrastructure	Spark, Airflow, BigQuery	FastAPI, Kubernetes, Redis

Worked example: Fraud detection: choosing the pattern

Transaction arrives: customer paying \$150 at a POS terminal. Is this fraud?

Decision tree:

Q1: Does the user need a response in <1 second? **Yes** (card is physically waiting).

Q2: Does the decision block a business transaction? **Yes** (payment must be approved or declined immediately).

⇒ **Online serving required.**

Contrast — promotional email targeting:

Q1: Does the user need a response in <1 second? **No** (email goes out tomorrow morning).

Q2: Does the decision block a business transaction? **No** (email is informational).

⇒ **Batch serving is sufficient and cheaper.**

✓ Key takeaway

Batch if the user can wait and data volume is large. Online if the decision must be made before the user moves on or a transaction completes.